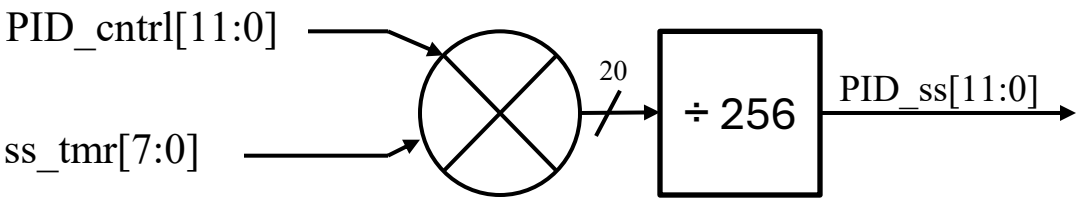
Exercise 07: Segway Math

- We are going to implement the math to compute **desired speed** going to the left and right motor

- A few features we want to embed into speed computation:
  - **Soft start**: so we don't "jerk" the platform level on power up
  - **Steering input**: allows us to turn
  - **Compensate for motor dead band**: allows small corrections to be consistent
  - **Over speed detection**: stops rider from leaning too far, beyond the control limits

Scaling with soft start

- The PID control signal should first be scaled by **ss_tmr**.
- This ensures a smooth start to the machine, so it does not “jerk” the platform level, but rather smoothly drives it to level on power up.
- **ss_tmr** slowly ramps up to a full 8-bit value after power up of the Segway.

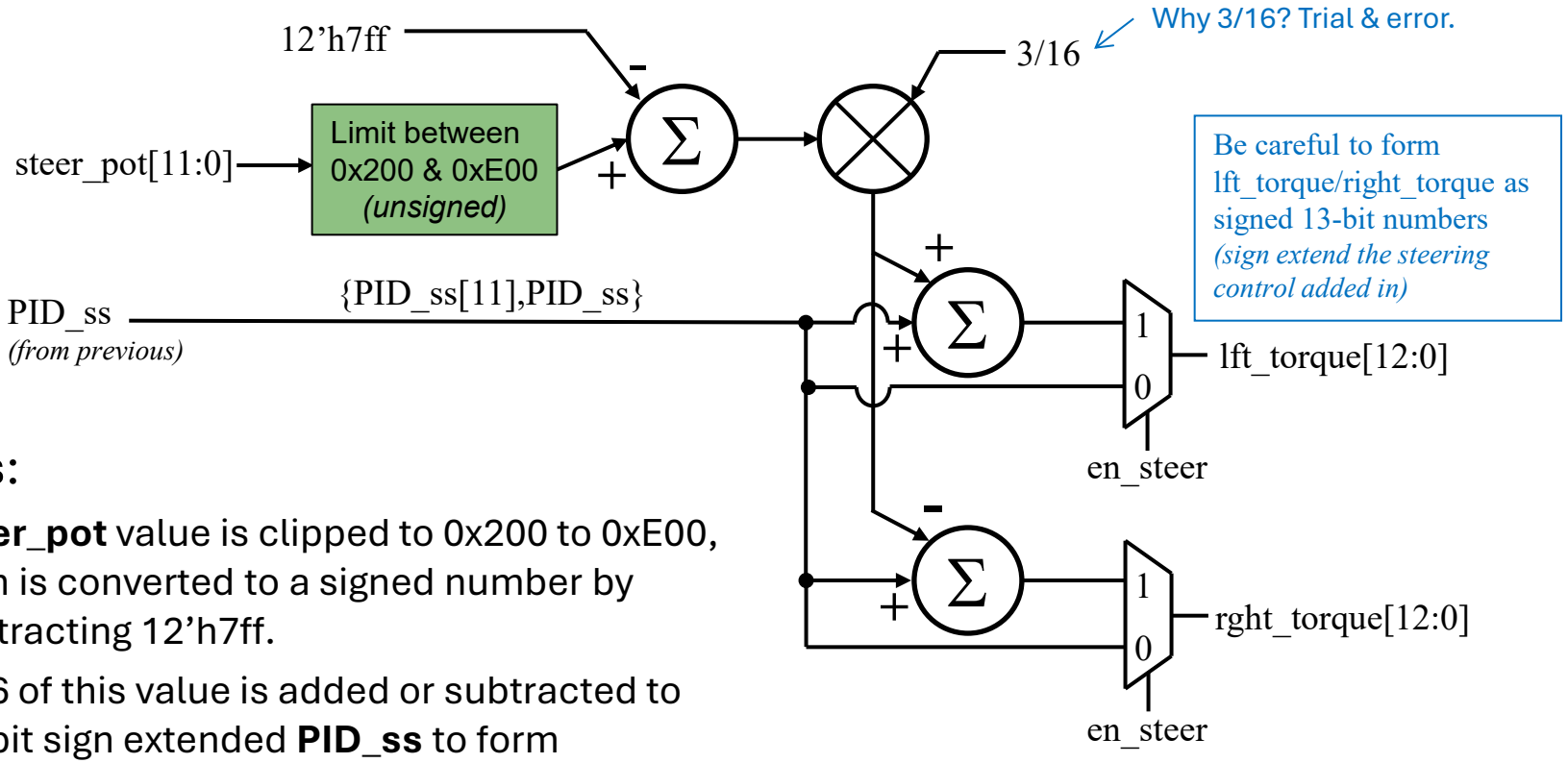


Notes:

- PID_cntrl is signed, and the multiply should be signed.
- **ss_tmr** is unsigned, so zero-extend to form a 9-bit quantity, and cast it to be signed for the multiplication ($signed({1’b0, ss_tmr})

Steering Input

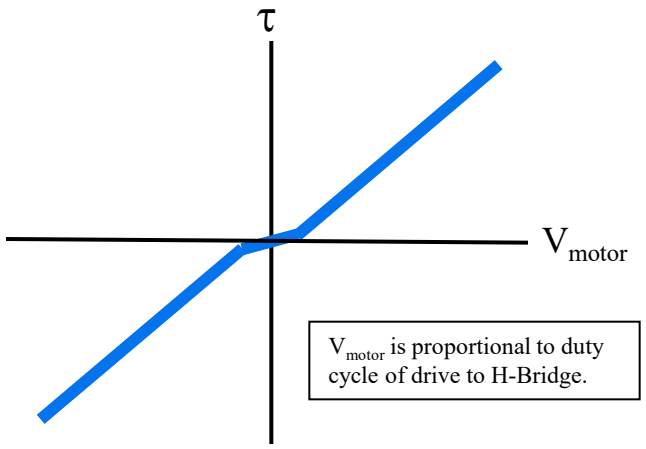
- The PID controller ensures the overall forward/reverse drive of the motors to keep the platform balanced.
- To effect steering a differential signal is added/subtracted to the left/right motors.
- This steering signal comes from a slide potentiometer that the rider controls.



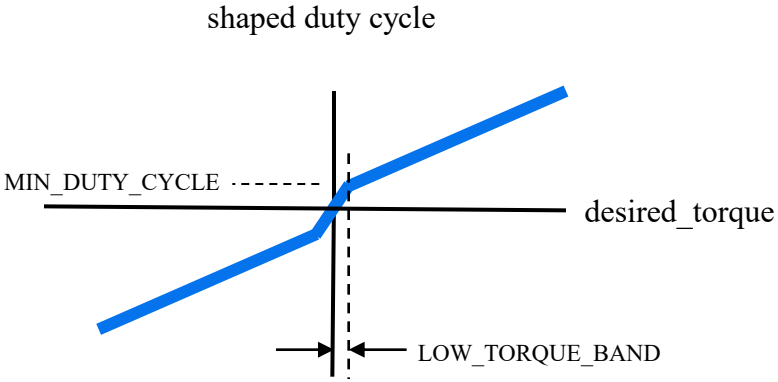
Notes:

- **steer_pot** value is clipped to 0x200 to 0xE00, then is converted to a signed number by subtracting 12'h7ff.
- 3/16 of this value is added or subtracted to 13-bit sign extended **PID_ss** to form **lft_torque/rght_torque** if steering is enabled.

DC Motor Dead Band



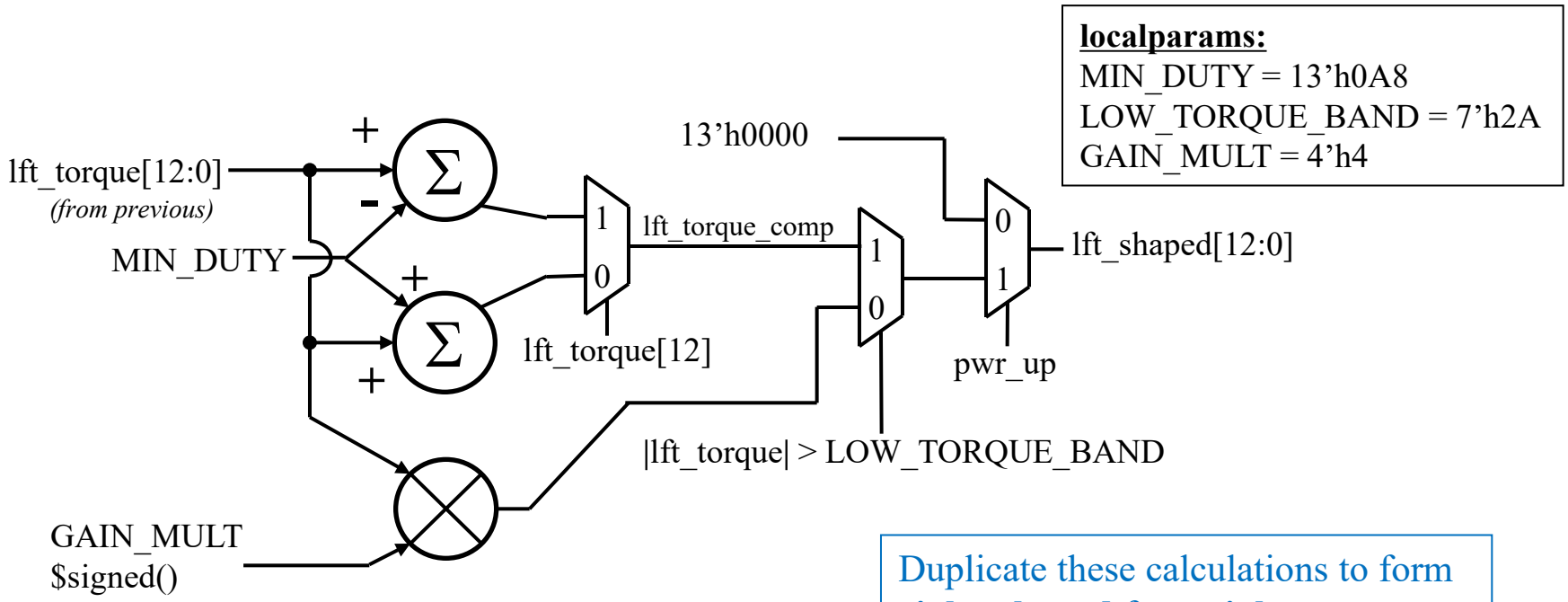
- Graph shows torque of a DC motor (τ) vs the voltage applied to the terminals of the motor (V_{motor})
- Notice the "dead band" in the middle? For example, we are using 30V motors, but they cannot overcome their own internal friction from -1.4V to +1.4V.
- With a voltage magnitude that exceeds 1.4V the torque & speed increases linearly with voltage.
- This deadband, though small, causes some havoc when small corrections are needed to keep the platform level. We need to compensate for it.



- We will scale/shape our desired torque to compensate for the DC motors dead zone and compress it into a small range of desired torques:
  (-LOW_TORQUE_BAND,LOW_TORQUE_BAND)
- When: **|desired_torque| < LOW_TORQUE_BAND,** we are in the steep part of the compensation and will scale the **desired_torque** by **GAIN_MULTIPLIER** (greater than unity).
- When **|desired_torque| ≥ LOW_TORQUE_BAND, desired_torque** will not be scaled up, but will have **MIN_DUTY_CYCLE** added to it if it is positive, or subtracted if it is negative.

Deadzone Shaping

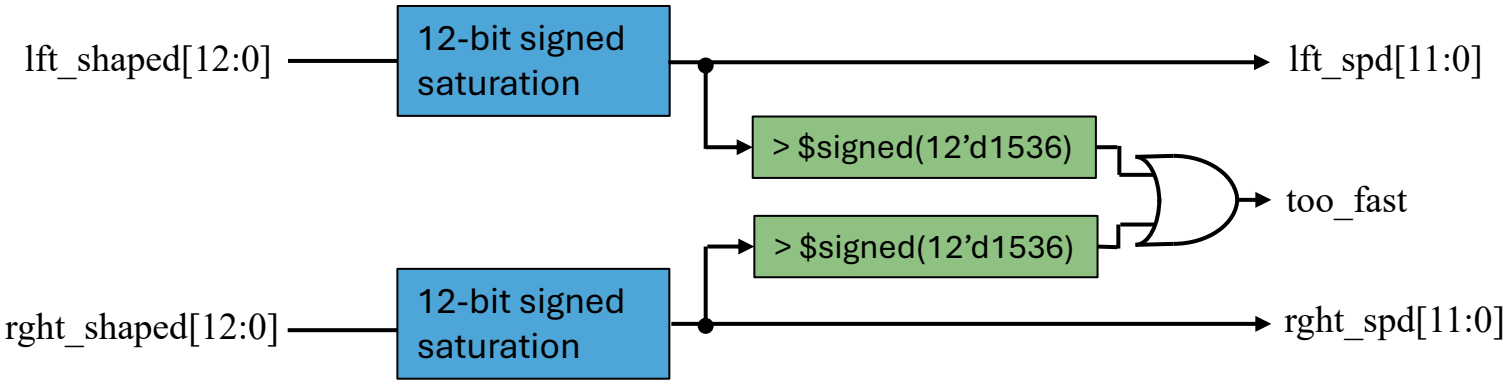
- We will use the following structure to perform the shaping computation
  - If **lft_toque** is negative (MSB set) then the compensated torque is: **lft_torque** - MIN_DUTY, otherwise it is **lft_torque** + MIN_DUTY.
  - If the abs(**lft_torque**) < LOW_TORQUE_BAND (i.e., we are inside the deadzone), we need to use **lft_torque***$signed(GAIN_MULT) to form the shaped torque.
  - Finaly if the Segway is not powered up, we set the shaped torque to zero.



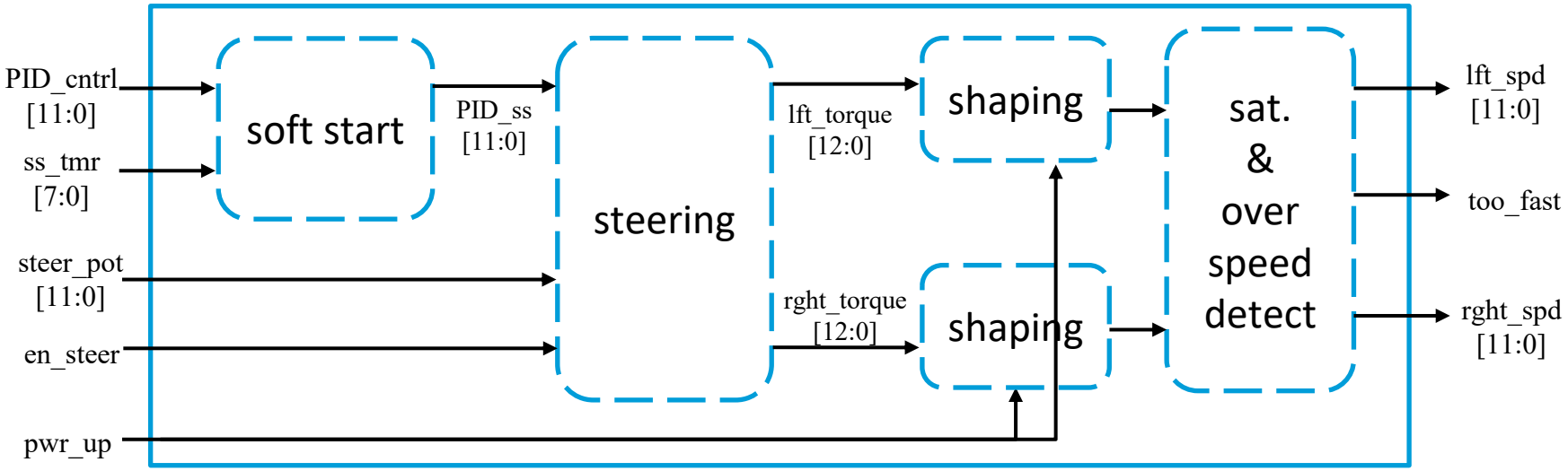
Duplicate these calculations to form **right_shaped** from **right_torque**

Saturation & Over Speed Detection

- Finally, the signed 12-bit signals **lft_spd** and **rght_spd** are formed by signed 12-bit saturation of **lft_shaped/right_shaped**.
- The signal **too_fast** is formed to inform the rider they are going forward too fast and pushing the control limits of the Segway.



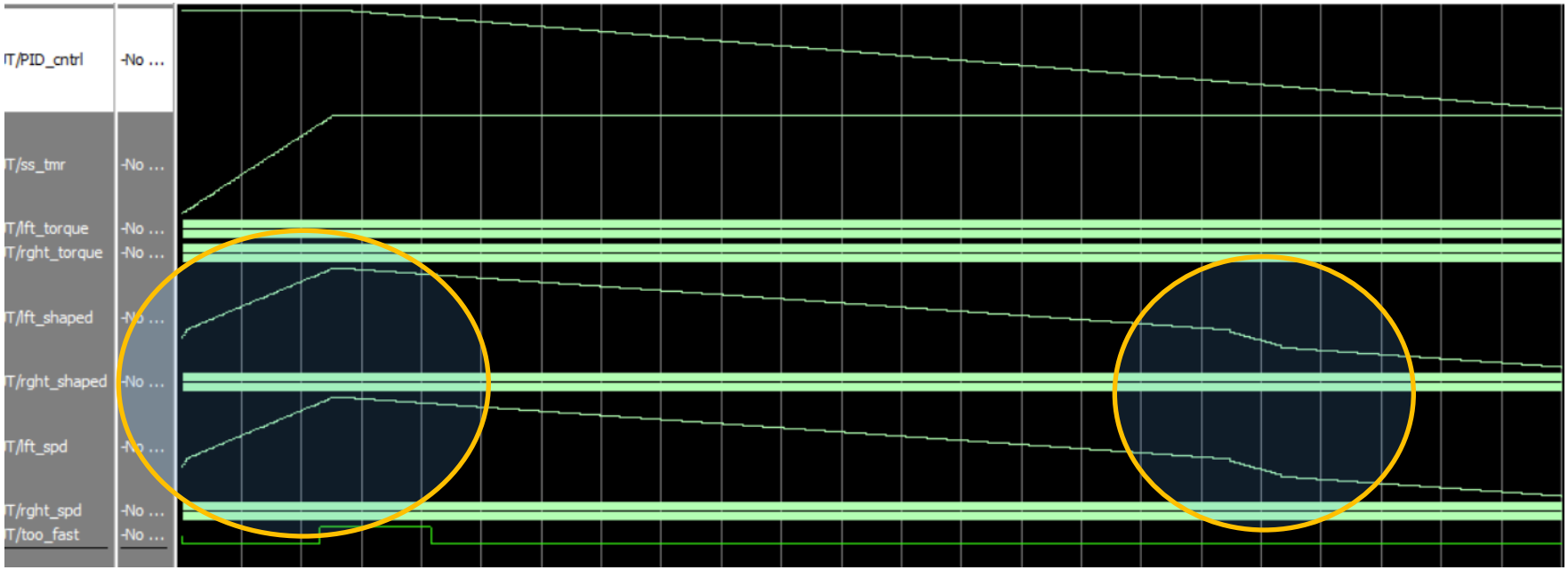
Ex07: Segway Math



Signal	Dir	Description
PID_cntrl[11:0]	in	Signed 12-bit control from PID that dictates frwrd/rev drive of motors to maintain platform balance
ss_tmr[7:0]	in	Unsigned 8-bit scaling quantity used to provide a soft-start to control loop. **PID_cntrl** is scaled by this timer that ramps up slowly from power on.
steer_pot[11:0]	in	12-bit unsigned measure of steering potentiometer. Comes from **A2D_intf**. Limited and converted to signed version internal to **Segway_math**
en_steer	in	Indicates steering has been enabled. Enabled by rider having equal weight distribution on load cells.
pwr_up	in	If ~**pwr_up** then both **lft_spd** & **rght_spd** are forced to zero
lft_spd/rght_spd[11:0]	out	Desired output speed/torque for each of left/right motors. 12-bit signed quantity.
too_fast	out	If either lft_spd or right_spd exceed 12'd1536 then this signal is asserted. Used to warn rider of approaching control limits.

Recommended Test 1

- **steer_en** = 0 and **pwr_up** = 1
- **PID_cntrl** starts at 12'h5FF, **ss_tmr** initially ramps from 0 to 8'hFF
- **PID_cntrl** ramps from 12'h5FF (+1535) to 12'hE00 (-512)

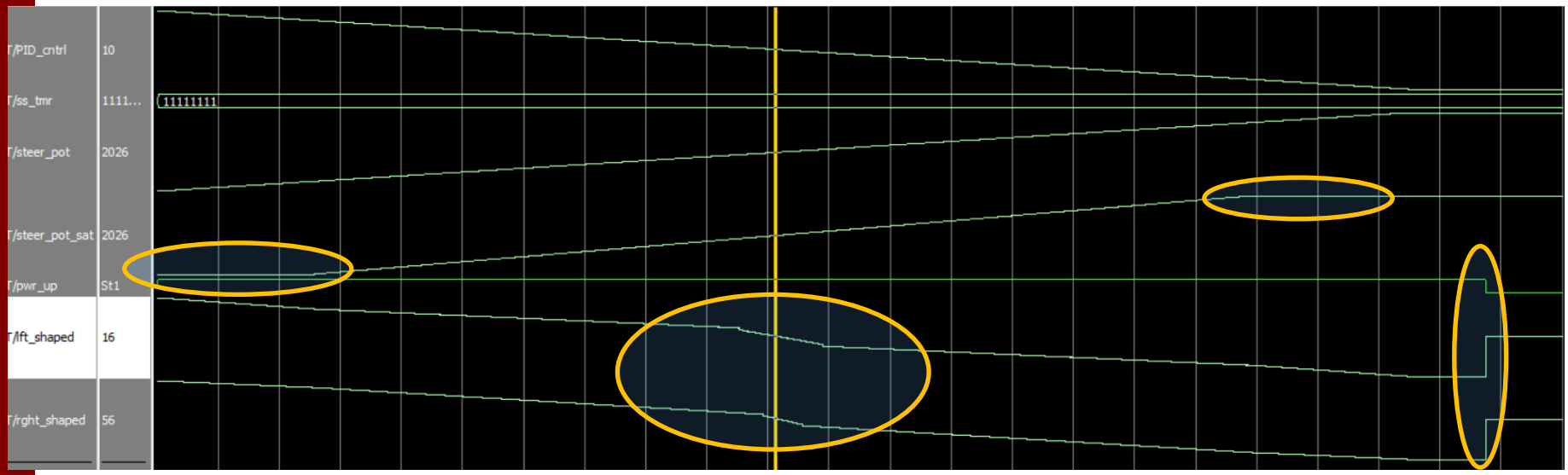


During initial ramp up of **ss_tmr** we see **lft_shaped/right_shaped** ramp up too. Initially in high gain region.
We also see **too_fast** asserted after **ss_tmr** is near full, and until **PID_cntrl** falls far enough.

As **PID_cntrl** crosses through LOW_TORQUE_BAND we see the shaped signal transition through the high gain region.

Recommended Test 2

- **steer_en** = 1 and **pwr_up** = 1
- **ss_tmr** stays at 8'hFF, **PID_cntrl** starts at 12'h3FF (1023) and is ramped down to 12'hC00 (-1024)
- **steer_pot** is ramped from 12'h000 to 12'hFFE (4094)
- **pwr_up** = 0 at the end



Note the saturation/clipping of **steer_pot_sat** to between 0x200 and 0xE00.

NOTE: *you may have a different signal name than* **steer_pot_sat**.

In this simulation we see difference between **lft** & **rght** because **en_steer=1** and **steer_pot** is being swept.

When **pwr_up** falls both lft/right spds go to zero

Testing SegwayMath

- These two tests are obviously not sufficient to prove your **SegwayMath** is good.
- However, if you can duplicate similar results, you are probably pretty close.
- We will revisit validation of this unit in the future when we learn more about testing with test vectors read from a file.
- For now, try to validate to the above two tests.

Submit: SegwayMath.sv and **SegwayMath_tb.sv** by end of class Wed.

Signed Operations in Verilog

- Addition and subtraction are agnostic operations
  - An adder to add 2 signed numbers is no different than an adder to add 2 unsigned numbers
- Multiplication is **not** agnostic
  - A multiplier for signed numbers is a little different than a multiplier for unsigned numbers.
- Magnitude comparison is **not** agnostic
- Right shifting is **not** agnostic (arithmetic vs logical)

- Multiplier will be signed if **all three** signals are declared as type signed

```
assign product = operand1 * operand2;
```

- Magnitude comparison will be signed if **both** numbers are signed

```
if (num1 > num2)
```

- >>> will be signed if **both** the dest signal and operand being shifted are signed

```
assign shifted = operand >>> 3;
```